0200. Number of Islands

Medium | Category: Graphs | LeetCode: https://leetcode.com/problems/number-

Problem

- In a grid of '1' (land) and '0' (water), count the islands (groups of land connected up/down/left/right).

Intuition

- Each unvisited land cell starts a new island. Flood-fill (DFS/BFS) from it to mark the whole island as visited.

Brute Force

- Union-Find over land cells also works in near-linear time; flood fill is simpler.

Optimal Approach

- For every cell: if '1', count++ and sink(r, c).
- Sink: if out of bounds or not '1' return; set '0'; recurse 4 directions.

Worked Example

```
11000
11000       island 1: top-left 2x2 block
00100       island 2: the single centre cell
00011       island 3: bottom-right pair
answer 3
```

Java Solution

```java
public class NumberOfIslands {
    public int numIslands(char[][] grid) {
        int count = 0;
        for (int r = 0; r < grid.length; r++)
            for (int c = 0; c < grid[0].length; c++)
                if (grid[r][c] == '1') { count++; sink(grid, r, c); }
        return count;
    }

    private void sink(char[][] g, int r, int c) {
        if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1') return;
        g[r][c] = '0';
        sink(g, r + 1, c); sink(g, r - 1, c); sink(g, r, c + 1); sink(g, r, c - 1);
    }
}
```

Complexity

- Time: O(m n). Space: O(m n) recursion in the worst case.

Pitfalls

- This version mutates the input; copy the grid if the caller needs it.
- Diagonal cells are not connected.